

Modulo_1

Back to: [00 Index Algoritmi](#)

Modulo 1 - Algoritmi e Strutture Dati

1. Strutture Dati Lineari

Una **struttura dati** è un modo sistematico e organizzato di memorizzare, gestire e accedere ai dati all'interno della memoria di un calcolatore, consentendo l'esecuzione efficiente di specifiche operazioni (ricerca, inserimento, cancellazione, scansione).

1.1 Array

Modello Teorico & Operazioni

L'**array** (o vettore) è una sequenza contigua e omogenea di elementi in memoria ad allocazione fissa o dinamica. Gli elementi sono indicizzati da 0 a $n-1$ (oppure da 1 a n).

- **Modello di indirizzamento in memoria:** ogni elemento è accessibile in tempo costante $O(1)$ calcolando il suo indirizzo fisico tramite la formula:

$$\text{Indirizzo}(A[i]) = \text{IndirizzoBase}(A) + i \cdot \text{sizeof}(\text{Tipo})$$

- **Operazioni fondamentali:**
 - `length`: restituisce la dimensione dell'array in tempo $O(1)$.
 - `get(r)`: restituisce l'elemento in posizione r ($O(1)$). Errore se $r < 0$ o $r > n-1$.
 - `set(r, e)`: sostituisce l'elemento in posizione r con e ($O(1)$).
 - `add(r, e)`: inserisce l'elemento e in posizione r , shiftando a destra gli elementi successivi ($O(n)$).
 - `remove(r)`: elimina l'elemento in posizione r , shiftando a sinistra gli elementi successivi per compattare ($O(n)$).
 - **Inizializzazione**: costo $\Theta(n)$ per l'allocazione e azzeramento dello spazio contiguo.
- **Array Dinamici (es. `std::vector`, `ArrayList`):** raddoppiano la capacità ($2 \times \text{capacità}$) quando saturi, garantendo un costo ammortizzato di $O(1)$ per l'inserimento in coda (`append`), pur con costo $O(n)$ nel caso peggiore di riallocazione.

Analisi della Complessità

Operazione	Tempo (Caso Migliore)	Tempo (Caso Peggior)	Spazio Ausiliario	Note / Spiegazione
Accesso (<code>get</code> / <code>set</code>)	$O(1)$	$O(1)$	$O(1)$	Accesso diretto tramite calcolo aritmetico dell'indice.
Inserimento (<code>add</code> in coda)	$O(1)$	$O(n)$ (dinamico)	$O(1)$ / $O(n)$	$O(1)$ ammortizzato su array dinamici; $O(n)$ se richiede riallocazione.
Inserimento (<code>add</code> intermedio/testa)	$O(1)$ (in coda)	$O(n)$	$O(1)$	Richiede lo shift a destra di tutti gli elementi successivi.
Cancellazione (<code>remove</code>)	$O(1)$ (in coda)	$O(n)$	$O(1)$	Richiede lo shift a sinistra per compattare lo spazio contiguo.
Creazione / Inizializzazione	$\Theta(n)$	$\Theta(n)$	$O(n)$	Alloca n celle contigue in memoria.
Ricerca (non ordinato)	$O(1)$	$O(n)$	$O(1)$	Scansione lineare sequenziale.
Ricerca (ordinato)	$O(1)$	$O(\log n)$	$O(1)$	Ricerca binaria (dicotomica).

1.2 Liste Concatenate (Linked Lists)

Modello Teorico & Operazioni

Una **lista concatenata** è una collezione dinamica di nodi memorizzati in locazioni di memoria non contigue, dove ciascun nodo contiene il campo dato (`element`) e i riferimenti puntatori ai nodi adiacenti (`next` , `prec` / `prev`).

- **Operazioni del TDA Lista:**

- `first()`: restituisce il primo elemento (testa) o `NIL` se vuota ($O(1)$).
- `last()`: restituisce l'ultimo elemento o `NIL` se vuota ($O(1)$ se si mantiene il puntatore di coda).
- `before(p)` / `after(p)`: restituisce la posizione del nodo precedente / successivo a p ($O(1)$).
- `isEmpty()`: verifica se la lista è vuota ($O(1)$).
- `size()`: restituisce il numero di elementi ($O(1)$ se mantenuto contatore, altrimenti $O(n)$).
- `insertAfter(p, e)`: inserisce un nuovo elemento e subito dopo la posizione p ($O(1)$).
- `insertBefore(p, e)`: inserisce un nuovo elemento e subito prima della posizione p ($O(1)$ in lista doppia).
- `remove(p)`: rimuove il nodo alla posizione p ($O(1)$ in lista doppia).

Pseudocodice Dettagliato (Convenzione Pinotti)

```
Algorithm InsertAfter(p, e):
```

```
    create a new nodo v
    v.element = e
    v.prec = p
    v.next = p.next
    (p.next).prec = v    // N.B.: l'ordine di riassegnamento dei puntatori
    p.next = v           // deve evitare la perdita del riferimento al nodo
    successivo
    return v
```

```
Algorithm InsertBefore(p, e):
```

```
    create a new nodo v
    v.element = e
    v.next = p
    v.prec = p.prec
    (p.prec).next = v
    p.prec = v
    return v
```

```
Algorithm Remove(L, p):
```

```
    if p.prec ≠ NIL then
        (p.prec).next = p.next
    else
        L.head = p.next
    if p.next ≠ NIL then
        (p.next).prec = p.prec
    dispose(p)
```

Analisi della Complessità

Operazione	Singola (Senza Coda)	Singola (Con Coda)	Doppiamente Concatenata	Spazio Ausiliario
Accesso per Indice (<i>i</i> -esimo)	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Ricerca di una Chiave	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Inserimento in Testa	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Inserimento in Coda	$O(n)$	$O(1)$	$O(1)$	$O(1)$
insertAfter(p, e) / insertBefore(p, e)	$O(1)$ (after), $O(n)$ (before)	$O(1)$ (after), $O(n)$ (before)	$O(1)$	$O(1)$
Cancellazione remove(p)	$O(n)$ (trova prec)	$O(n)$	$O(1)$	$O(1)$

1.3 Liste ad Accesso Limitato: Pile (Stack) e Code (Queue)

Pile (Stack - LIFO)

La **pila** è una struttura dati lineare basata sulla disciplina **LIFO** (*Last-In, First-Out*): l'ultimo elemento inserito è il primo ad essere estratto.

- **Operazioni:**
 - `push(S, x)`: inserisce l'elemento x in cima alla pila ($O(1)$).
 - `pop(S)`: rimuove e restituisce l'elemento in cima ($O(1)$); solleva errore di *Stack Underflow* se vuota.
 - `top(S)` / `peek(S)`: restituisce il valore in cima senza rimuoverlo ($O(1)$).
 - `isEmpty(S)`: restituisce `true` se la pila è vuota ($O(1)$).

```
Algorithm Push(S, x):
```

```
    S.top = S.top + 1
    S[S.top] = x
```

```
Algorithm Pop(S):
```

```
    if IsEmpty(S) then
        error "Stack Underflow"
    x = S[S.top]
```

```
S.top = S.top - 1  
return x
```

Code (Queue - FIFO) e Code Doppie (Deque)

La **coda** è una struttura dati lineare basata sulla disciplina **FIFO** (*First-In, First-Out*): il primo elemento inserito è il primo ad essere estratto.

- **Operazioni:**
 - `enqueue(Q, x)`: inserisce l'elemento x in fondo alla coda ($O(1)$).
 - `dequeue(Q)`: rimuove e restituisce l'elemento in testa ($O(1)$); solleva errore di *Queue Underflow* se vuota.
 - `isEmpty(Q)`: verifica se la coda è vuota ($O(1)$).
- **Implementazione con Array Circolare:**
 - Si mantengono due indici `head` e `tail` con incremento modulare modulo N :

$$\text{tail} = (\text{tail} + 1) \bmod N, \quad \text{head} = (\text{head} + 1) \bmod N$$

Tutte le operazioni operano in tempo strettamente $\Theta(1)$ nel caso peggiore.

2. Studio della Complessità e Notazione Asintotica

2.1 Notazioni Asintotiche e Ordini di Grandezza

L'analisi teorica prescinde dalla macchina fisica e adotta il modello di costo **RAM** (*Random Access Machine*), dove ogni operazione primitiva richiede costo $O(1)$ e l'efficienza è espressa asintoticamente al crescere della dimensione dell'input $n \rightarrow \infty$.

Definizioni Formali (O , Ω , Θ , o , ω)

Siano $f(n), g(n) \geq 0$ funzioni asintoticamente positive definite sui numeri naturali:

1. Notazione O (Limite Asintotico Superiore / Upper Bound):

$$O(g(n)) = \{f(n) : \exists c > 0, n_0 \in \mathbb{N} \text{ tale che } 0 \leq f(n) \leq c \cdot g(n), \forall n \geq n_0\}$$

- **Criterio del Limite:** Se $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = L < \infty$, allora $f(n) \in O(g(n))$.

2. Notazione Ω (Limite Asintotico Inferiore / Lower Bound):

$$\Omega(g(n)) = \{f(n) : \exists c > 0, n_0 \in \mathbb{N} \text{ tale che } 0 \leq c \cdot g(n) \leq f(n), \forall n \geq n_0\}$$

- **Criterio del Limite:** Se $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} > 0$ (anche $+\infty$), allora $f(n) \in \Omega(g(n))$.

3. Notazione Θ (Limite Asintotico Stretto / Tight Bound):

$$\Theta(g(n)) = \{f(n) : \exists c_1 > 0, c_2 > 0, n_0 \in \mathbb{N} \text{ tale che } 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n), \forall n \geq n_0\}$$

- **Teorema Fondamentale:** $f(n) \in \Theta(g(n)) \iff f(n) \in O(g(n)) \wedge f(n) \in \Omega(g(n))$.
- **Criterio del Limite:** Se $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = L$ con $0 < L < \infty$, allora $f(n) \in \Theta(g(n))$.

4. Notazione o (Piccolo-o / Limite Superiore Non Stretto):

$$o(g(n)) = \{f(n) : \forall c > 0, \exists n_0 \in \mathbb{N} \text{ tale che } 0 \leq f(n) < c \cdot g(n), \forall n \geq n_0\} \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

5. Notazione ω (Piccolo-omega / Limite Inferiore Non Stretto):

$$\omega(g(n)) = \{f(n) : \forall c > 0, \exists n_0 \in \mathbb{N} \text{ tale che } 0 \leq c \cdot g(n) < f(n), \forall n \geq n_0\} \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

Proprietà e Relazioni tra Ordini di Grandezza

- **Transitività:** $f(n) \in \Theta(g(n)) \wedge g(n) \in \Theta(h(n)) \implies f(n) \in \Theta(h(n))$ (vale anche per O, Ω, o, ω).
- **Simmetria:** $f(n) \in \Theta(g(n)) \iff g(n) \in \Theta(f(n))$.
- **Simmetria Trasposta:** $f(n) \in O(g(n)) \iff g(n) \in \Omega(f(n))$ e $f(n) \in o(g(n)) \iff g(n) \in \omega(f(n))$.
- **Gerarchia di Crescita Standard:**

$$1 \ll \log \log n \ll \log n \ll \sqrt{n} \ll n \ll n \log n \ll n^2 \ll n^c \ll 2^n \ll c^n \ll n! \ll n^n$$

2.2 Costo delle Operazioni di Base e Strutture di Controllo

- **Operazioni Primitive:** assegnamenti (`x = a`), letture/scritture in memoria, test condizionali elementari $\implies \Theta(1)$.
- **Blocchi Condizionali (if-then-else):**

$$T(n) = T_{\text{guardia}}(n) + \max(T_{\text{then}}(n), T_{\text{else}}(n))$$

- **Cicli Semplici e Annidati:** si traducono nel calcolo della sommatoria dei costi delle singole iterazioni.

2.3 Analisi di Codici Iterativi

Modello Teorico & Invarianti di Ciclo

Un **invariante di ciclo** è una proprietà formale dimostrata per induzione:

1. **Inizializzazione:** vera prima della prima iterazione.
2. **Conservazione:** se vera prima dell'iterazione k , resta vera prima della $k+1$.
3. **Conclusione:** all'uscita dal ciclo, garantisce la correttezza del risultato.

Esempi Risolti Tipici d'Esame

Esempio 1: Calcolo di Medie dei Prefissi (*Prefix Averages*)

- Versione non ottima (quadratica): ricalcola la somma da 1 a i per ciascun indice $\implies \sum_{i=1}^n i = \Theta(n^2)$.
- Versione ottima (lineare): mantiene un accumulatore $S \implies \Theta(n)$.

Esempio 2: Cicli con Passo Moltiplicativo

```
for i = 1 to n do
  j = 1
  while j < n do
    j = j * 2
```

Il ciclo interno compie $\lfloor \log_2 n \rfloor + 1$ iterazioni. Costo totale:
 $\sum_{i=1}^n \Theta(\log n) = \Theta(n \log n)$.

3. Equazioni di Ricorrenza e Master Theorem

3.1 Tecniche di Risoluzione delle Ricorrenze

1. Metodo di Sostituzione (Induzione)

1. Ipotizzare la forma della soluzione asintotica.
2. Dimostrare per induzione matematica determinando le costanti $c > 0$ e n_0 .
• **Trucco:** se compare un residuo $+d$, sottrarre un termine di ordine inferiore all'ipotesi induttiva: $T(n) \leq cn - b$.

2. Metodo dell'Albero di Ricorsione

Sviluppa l'albero calcolando:

- Costo a ciascun nodo al livello i : $f(n/b^i)$.
- Numero di nodi al livello i : a^i .
- Costo totale per livello: $a^i f(n/b^i)$.
- Altezza dell'albero: $h = \log_b n$.
- Costo totale: $T(n) = \sum_{i=0}^{\log_b n - 1} a^i f(n/b^i) + \Theta(n^{\log_b a})$.

Livello 0:	$f(n)$	$\rightarrow f(n)$
Livello 1:	$f(n/b)$ $f(n/b)$	$\rightarrow a \cdot$
Livello 2:	$f(n/b^2)$ $f(n/b^2)$ $f(n/b^2)$ $f(n/b^2)$	$\rightarrow a^2 \cdot$
Livello $\log_b(n)$:	$\dots \dots \dots \dots$	$\theta(1) \rightarrow$
	$\theta(1) \theta(1) \dots$	$\theta(1)$
	$\theta(n^{\log_b a})$	

		Totale:
Somma di tutti i livelli		

3. Metodo per Iterazione (Srotolamento)

Si sviluppa la ricorrenza k volte fino a ricavare l'espressione generale in funzione di k , ponendo poi la dimensione pari al caso base (es. $n/2^k = 1 \Rightarrow k = \log_2 n$).

3.2 Master Theorem (Teorema Esperto)

Si applica a ricorrenze nella forma canonica:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n), \quad a \geq 1, b > 1$$

Si confronta la funzione $f(n)$ con il termine spartiacque $n^{\log_b a}$:

Confronto tra $f(n)$ e $n^{\log_b a}$		
	+-----+	
++		
Caso 1	Caso 2	
Caso 3		
$f(n) = O(n^{\log_b(a) - \epsilon})$	$f(n) = \theta(n^{\log_b a})$	$f(n) =$
$\Omega(n^{\log_b(a) + \epsilon})$		
$(\epsilon > 0)$		$(\epsilon$
$> 0)$		
Dominano le Foglie	Costo Distribuito	Domina
la Radice		
		(se
$a \cdot f(n/b) \leq c \cdot f(n)$		
v	v	
v		

$$T(n) = \Theta(n^{\log_b a})$$

$$T(n) = \Theta(f(n))$$

$$T(n) = \Theta(n^{\log_b a} \cdot \log n)$$

1. **Caso 1 ($f(n) < n^{\log_b a}$ polinomialmente):** Se $f(n) = O(n^{\log_b a - \epsilon})$ per $\epsilon > 0 \implies T(n) = \Theta(n^{\log_b a})$.
2. **Caso 2 ($f(n) \approx n^{\log_b a}$):** Se $f(n) = \Theta(n^{\log_b a}) \implies T(n) = \Theta(n^{\log_b a} \log n)$. *Estensione con termini polilogaritmici:* Se $f(n) = \Theta(n^{\log_b a} \log^k n)$ con $k \geq 0 \implies T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.
3. **Caso 3 ($f(n) > n^{\log_b a}$ polinomialmente):** Se $f(n) = \Omega(n^{\log_b a + \epsilon})$ per $\epsilon > 0$ e vale la *condizione di regolarità* $af(n/b) \leq cf(n)$ con $c < 1 \implies T(n) = \Theta(f(n))$.

Guida Operativa in 3 Passi per gli Esercizi:

1. Identificare i parametri a , b e la funzione $f(n)$.
2. Calcolare l'esponente critico $E = \log_b a$ e scrivere la funzione spartiacque $n^{\log_b a}$.
3. Confrontare $f(n)$ con $n^{\log_b a}$ e determinare il caso applicabile.

4. Paradigma Divide et Impera e Algoritmi di Ricerca

4.1 Il Paradigma Divide et Impera

Il paradigma si articola in tre fasi:

1. **Divide:** suddivide il problema in sottoproblemi disgiunti di dimensione ridotta.
2. **Impera (Conquer):** risolve i sottoproblemi ricorsivamente (o direttamente se caso base).
3. **Combina:** unisce le soluzioni dei sottoproblemi per produrre la soluzione globale.

4.2 Algoritmi di Ricerca in Vettori

1. Ricerca Lineare Ricorsiva (`CercoValore`)

Scorre ricorsivamente l'array non ordinato da p a r . Tempo nel caso peggiore $\Theta(n)$.

```
Algorithm CercoValore(A, p, r, val):
    if p ≤ r then
        if A[p] = val then
            return p
        else
```

```
        return CercoValore(A, p + 1, r, val)
    else
        return -1
```

2. Ricerca Binaria (Dicotomica)

Richiede array **ordinato**. Dimezza lo spazio di ricerca ad ogni iterazione: $T(n) = T(n/2) + \Theta(1) \implies \Theta(\log n)$.

```
Algorithm BinarySearch(A, p, r, val):
    if p ≤ r then
        q = p + (r - p) / 2
        if A[q] = val then
            return q
        else if A[q] > val then
            return BinarySearch(A, p, q - 1, val)
        else
            return BinarySearch(A, q + 1, r, val)
    return -1
```

3. Ricerca di Prima e Ultima Occorrenza (Cluster di Chiavi)

Dato un array ordinato con elementi duplicati consecutivi (cluster):

```
Algorithm FirstOccorrenza(A, p, r, val):
    if (r - p + 1 > 3) then
        q = floor((p + r) / 2)
        if A[q] ≥ val then
            return FirstOccorrenza(A, p, q, val)
        else
            return FirstOccorrenza(A, q + 1, r, val)
    else
        j = p
        while j ≤ r and A[j] ≠ val do
            j = j + 1
        if j > r then return -1 else return j

Algorithm LastOccorrenza(A, p, r, val):
    if (r - p + 1 > 3) then
        q = floor((p + r) / 2)
        if A[q] ≤ val then
            return LastOccorrenza(A, q, r, val)
        else
            return LastOccorrenza(A, p, q - 1, val)
    else
        j = r
        while j ≥ p and A[j] ≠ val do
            j = j - 1
        if j < p then return -1 else return j
```

- **Conteggio Occorrenze nel Cluster:**

$\text{Occorrenze}(val) = \text{LastOccorrenza}(A, 1, n, val) - \text{FirstOccorrenza}(A, 1, n, val) + 1$,
calcolabile in tempo $O(\log n)$.

4.3 Ricerca di Massimo e Minimo

1. Calcolo del Massimo Divide et Impera (TrovaMax e TrovaMax-Pos)

```
Algorithm TrovaMax(A, p, r):
    if p < r then
        q = floor((p + r) / 2)
        lm = TrovaMax(A, p, q)
        rm = TrovaMax(A, q + 1, r)
        if lm > rm then return lm else return rm
    else
        return A[p]

Algorithm TrovaMax-Pos(A, p, q):
    if p == q then
        return (A[p], p)
    else
        z = floor((p + q) / 2)
        (lm, lpos) = TrovaMax-Pos(A, p, z)
        (rm, rpos) = TrovaMax-Pos(A, z + 1, q)
        if lm > rm then return (lm, lpos) else return (rm, rpos)
```

- **Complessità:** $T(n) = 2T(n/2) + O(1) \implies \Theta(n)$.

2. Minimo e Massimo Simultaneo (Algoritmo a Coppie)

Per trovare contemporaneamente il minimo e il massimo minimizzando i confronti:

- **Inizializzazione:**
 - Se n è **pari**: confronta $A[1]$ e $A[2]$, assegna il maggiore a **max** e il minore a **min** (1 confronto).
 - Se n è **dispari**: imposta sia **min** che **max** pari ad $A[1]$ (0 confronti).
- **Elaborazione a coppie:** per ciascuna coppia successiva $(A[i], A[i+1])$ si eseguono 3 confronti per 2 elementi:
 1. Si confronta $A[i]$ con $A[i+1]$ (1 confronto).
 2. Il vincitore si confronta con **max** corrente (1 confronto).
 3. Il perdente si confronta con **min** corrente (1 confronto).
- **Numero totale di confronti:** al più $\lceil \frac{3n}{2} \rceil - 2 = 3\lfloor n/2 \rfloor$ confronti (strettamente ottimale).

3. Massimo e Secondo Massimo (Prim-Sec / Torneo)

```

Algorithm Prim-Sec(A, p, r):
    if (p == r - 1) then // Caso base a 2 elementi
        if A[p] < A[p + 1] then
            return (A[p + 1], A[p])
        else
            return (A[p], A[p + 1])
    else
        q = floor((p + r) / 2)
        (Fl, Sl) = Prim-Sec(A, p, q)
        (Fr, Sr) = Prim-Sec(A, q + 1, r)
        if Fl < Fr then
            F = Fr
            S = max(Fl, Sr)
        else
            F = Fl
            S = max(Fr, Sl)
        return (F, S)

```

- **Complessità:** $\Theta(n)$. Tramite albero di torneo sono sufficienti $n + \lceil \log_2 n \rceil - 2$ confronti.

4. Minimo in Sequenze Unimodali (Min-Unimodale)

Per sequenze **prima strettamente decrescenti e poi crescenti** (convesse):

```

Algorithm Min-Unimodale(A, p, r):
    if (r - p + 1 > 3) then
        q = floor((p + r) / 2)
        if A[q] < A[q + 1] then
            return Min-Unimodale(A, p, q)
        else
            return Min-Unimodale(A, q + 1, r)
    else
        j = p
        while (j < r) and (A[j] > A[j + 1]) do
            j = j + 1
        return j

```

- **Complessità:** $T(n) = T(n/2) + O(1) \implies \Theta(\log n)$.

4.4 Ricerca in Matrici con Righe e Colonne Ordinate

Data una matrice $M[1..m, 1..n]$ in cui $M[i][j] \leq M[i+1][j]$ e $M[i][j] \leq M[i][j+1]$:

- **Saddleback Search (Step-wise Elimination):**
 1. Partenza da riga $i = 1$, colonna $j = n$ (angolo in alto a destra).
 2. Se $M[i][j] == x \implies$ trovato.
 3. Se $M[i][j] > x \implies$ elimina colonna j ($j = j - 1$).

4. Se $M[i][j] < x \implies$ elimina riga i ($i = i + 1$).

- **Complessità:** $\Theta(m + n)$ tempo, $O(1)$ spazio ausiliario.

4.5 Conteggio delle Inversioni (*Counting Inversions*)

Dato un array $A[1..n]$, una coppia di indici (i, j) è detta **inversione** se $i < j$ e $A[i] > A[j]$.

- **Spazio delle soluzioni:** il numero totale di inversioni è un intero compreso nell'intervallo $[0, \frac{n(n-1)}{2}]$.
 - 0 inversioni se l'array è già ordinato in senso crescente.
 - $\frac{n(n-1)}{2}$ inversioni se l'array è ordinato in senso decrescente.

```
Array A = [1, 8, 2, 7, 1, 5, 6, 3]
Inversioni generate dall'elemento A[2] = 8:
(2 < 3 e A[2]=8 > A[3]=2) → Inversione!
(2 < 4 e A[2]=8 > A[4]=7) → Inversione!
...
```

1. Algoritmo Brute-Force $\Theta(n^2)$

Verifica tutte le coppie di indici tramite un doppio ciclo `for`:

```
Algorithm BruteForceCountInv(A, n):
    inv = 0
    for i = 1 to n - 1 do
        for j = i + 1 to n do
            if A[i] > A[j] then
                inv = inv + 1
    return inv
```

- Numero di confronti eseguiti: $\sum_{i=1}^{n-1} (n - i) = \frac{n(n-1)}{2} = \Theta(n^2)$.

2. Variante BubbleSort (Counter-inversioni-BubbleSort)

```
Algorithm Counter-inversioni-BubbleSort(A, p, r):
    inv = 0
    for j = p to r do
        for t = r down to j + 1 do
            if A[t] < A[t - 1] then
                swap(A[t], A[t - 1])
                inv = inv + 1
    return inv
```

3. Algoritmo Divide et Impera Ottimo $\Theta(n \log n)$ (COUNTINV & INVMERGE)

Adotta lo schema di MergeSort: divide l'array a metà, calcola ricorsivamente le inversioni a sinistra (INVSX), a destra (INVDX) e quelle "a cavallo" durante la fusione (MINV):

```
Algorithm COUNTINV(A, i, f):
    if (f - i + 1) > 1 then
        q = floor((i + f) / 2)
        INVSX = COUNTINV(A, i, q)
        INVDX = COUNTINV(A, q + 1, f)
        MINV = INVMERGE(A, i, q, f)
        return INVSX + INVDX + MINV
    else
        return 0

Algorithm INVMERGE(A, i, q, f):
    nL = q - i + 1
    nR = f - q
    create arrays L[1..nL] = A[i..q] and R[1..nR] = A[q+1..f]
    l = 1, r = 1, k = i, tmp = 0

    while (l ≤ nL) and (r ≤ nR) do
        if L[l] ≤ R[r] then
            A[k] = L[l]
            l = l + 1
        else
            A[k] = R[r]
            r = r + 1
            tmp = tmp + (nL - l + 1) // Poiché L è ordinato, R[r] è minore
di tutti gli (nL - l + 1) elementi rimanenti in L!
            k = k + 1

    while (l ≤ nL) do
        A[k] = L[l]
        l = l + 1, k = k + 1
    while (r ≤ nR) do
        A[k] = R[r]
        r = r + 1, k = k + 1

    return tmp
```

- **Relazione di Ricorrenza:** $T(n) = 2T(n/2) + \Theta(n) \implies \Theta(n \log n)$.

4.6 Tabella di Riepilogo Complessità Ricerche

Algoritmo	Caso Migliore	Caso Peggior	Caso Medio	Spazio Ausiliario
CercoValore (Lineare)	$O(1)$	$\Theta(n)$	$\Theta(n)$	$O(n)$ stack

Algoritmo	Caso Migliore	Caso Peggior	Caso Medio	Spazio Ausiliario
BinarySearch (Ricerca Binaria)	$O(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$O(1) / O(\log n)$
FirstOccorrenza / LastOccorrenza	$O(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$O(\log n)$
TrovaMax (Divide et Impera)	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$O(\log n)$
Minimo e Massimo Simultaneo	$\Theta(n)$	$3\lfloor n/2 \rfloor$ conf.	$\Theta(n)$	$O(1)$
Prim-Sec (Max e 2° Max)	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$O(\log n)$
Min-Unimodale	$O(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$O(\log n)$
Ricerca Matrice Ordinata	$O(1)$	$\Theta(m + n)$	$\Theta(m + n)$	$O(1)$
COUNTINV (Conteggio Inversioni)	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$

5. Algoritmi di Ordinamento Basati su Confronti

5.1 Algoritmi di Ordinamento Incrementali ed Elementari

1. Insertion Sort

```

Algorithm InsertionSort(A, n):
  for j = 2 to n do
    key = A[j]
    i = j - 1
    while i > 0 and A[i] > key do
      A[i + 1] = A[i]
      i = i - 1
    A[i + 1] = key

```

- **Complessità:** Caso Migliore $\Theta(n)$ (già ordinato), Peggior $\Theta(n^2)$, Medio $\Theta(n^2)$.
- **Stabile:** Sì | **In-Place:** Sì.

2. ReiterateSort / Selection Sort

```

Algorithm ReiterateSort(A, p, r):
  for j = r down to p do
    (val, index) = TrovaMax-Pos(A, p, j)
    swap(A[j], A[index])

```

- **Complessità:** $\Theta(n^2)$ confronti sia nel caso migliore che peggiore; esegue al più n scambi.
- **Stabile:** No | **In-Place:** Sì.

3. Bubble Sort

```
Algorithm BubbleSort(A, p, r):
  for j = p to r do
    t = r
    while t > j do
      if A[t] < A[t - 1] then
        swap(A[t], A[t - 1])
      t = t - 1
```

- **Complessità:** $\Theta(n^2)$ (riducibile a $\Theta(n)$ nel caso migliore con flag `swapped`).
- **Stabile:** Sì | **In-Place:** Sì.

5.2 Merge Sort e Varianti

1. Merge Sort Standard (2-Way)

```
Algorithm MergeSort(A, p, r):
  if p < r then
    q = floor((p + r) / 2)
    MergeSort(A, p, q)
    MergeSort(A, q + 1, r)
    Merge(A, p, q, r)
```

- **Ricorrenza:** $T(n) = 2T(n/2) + \Theta(n) \implies \Theta(n \log n)$ in tutti i casi.
- **Spazio Ausiliario:** $\Theta(n)$ | **Stabile:** Sì | **In-Place:** No.

2. 3MergeSort (Merge Sort a 3 Vie)

Divide l'array in tre parti di dimensione $n/3$:

```
Algorithm 3MergeSort(A, p, r):
  if (r - p + 1 ≥ 3) then
    q1 = p + floor((r - p) / 3)
    q2 = p + 2 * floor((r - p) / 3)
    3MergeSort(A, p, q1)
    3MergeSort(A, q1 + 1, q2)
    3MergeSort(A, q2 + 1, r)
    Merge3(A, p, q1, q2, r)
  else
    InsertionSort(A, p, r)
```

- **Ricorrenza:** $T(n) = 3T(n/3) + cn \implies \Theta(n \log_3 n) = \Theta(n \log n)$.

3. MergeInsSort (Algoritmo Ibrido / Timsort Style)

```
Algorithm MergeInsSort(A, p, r):
  if p < r then
    if (r - p + 1 ≤ 3) then
      InsertionSort(A, p, r)
    else
      q = floor((p + r) / 2)
      MergeInsSort(A, p, q)
      MergeInsSort(A, q + 1, r)
      Merge(A, p, q, r)
```

- **Complessità:** $\Theta(n \log n)$.

5.3 QuickSort

QuickSort adotta il partizionamento attorno a un pivot. È uno dei più veloci algoritmi in pratica grazie a costanti moltiplicative molto ridotte e localizzazione di memoria (*cache friendly*).

```
Algorithm QuickSort(A, p, r):
  if p < r then
    q = Partition(A, p, r)
    QuickSort(A, p, q - 1)
    QuickSort(A, q + 1, r)
```

Procedure di Partizione: Lomuto vs Hoare

Lomuto Partitioning:

[	≤ Pivot		> Pivot		Non Esplorato		Pivot	]
p		i	i+1		j-1 j		r-1 r	

Hoare Partitioning:

[	≤ Pivot		Non Esplorato		≥ Pivot	]
p		i→		←j		r

Partizione di Lomuto (Pivot in $A[r]$)

```
Algorithm Partition-Lomuto(A, p, r):
  x = A[r]
  i = p - 1
  for j = p to r - 1 do
    if A[j] ≤ x then
      i = i + 1
      swap(A[i], A[j])
```

```
swap(A[i + 1], A[r])
return i + 1
```

Partizione di Hoare (Due puntatori convergenti)

```
Algorithm Partition-Hoare(A, p, r):
    pivot = A[p]
    i = p - 1
    j = r + 1
    while true do
        repeat j = j - 1 until A[j] ≤ pivot
        repeat i = i + 1 until A[i] ≥ pivot
        if i < j then
            swap(A[i], A[j])
        else
            return j
```

Analisi della Complessità e Sbilanciamento delle Partizioni

- **Caso Peggior:** Partizionamento massimamente sbilanciato (0 e $n-1$ elementi):

$$T(n) = T(n-1) + T(0) + \Theta(n) = T(n-1) + \Theta(n) = \Theta(n^2)$$

- **Caso Migliore:** Partizionamento perfettamente bilanciato ($n/2$ e $n/2$):

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$$

- **Partizionamento Quasi-Bilanciato (es. Split 9 a 1):**

$$T(n) = T(9n/10) + T(n/10) + cn$$

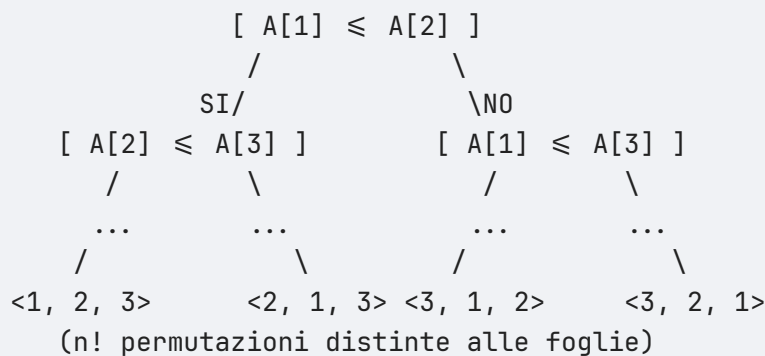
L'albero di ricorsione ha profondità compresa tra $\log_{10} n$ e $\log_{10/9} n = O(\log n)$, con ogni livello completo avente costo cn . Il costo complessivo rimane strettamente $\Theta(n \log n)$.

- **Intuizione per il Caso Medio:** anche se i livelli dell'albero alternano livelli "buoni" (bilanciati) e livelli "cattivi" (sbilanciati), il costo extra raddoppia solo la costante nascosta nella notazione asintotica $\Rightarrow \Theta(n \log n)$ atteso.
- **Randomized-QuickSort:** sceglie il pivot uniformemente a caso tra gli indici $[p..r]$, garantendo tempo atteso $O(n \log n)$ indipendente dalla distribuzione dell'input.
- **Spazio Ausiliario:** $O(\log n)$ per lo stack ricorsivo nel caso migliore/medio (con ricorsione in coda sul ramo minore), $O(n)$ nel caso peggiore.
- **Stabile:** No | **In-Place:** Sì.

5.4 Limite Inferiore all'Ordinamento e Albero di Decisione

Teorema del Limite Inferiore $\Omega(n \log n)$

Ogni algoritmo di ordinamento basato esclusivamente su confronti richiede, nel caso peggiore, $\Omega(n \log n)$ confronti.



Dimostrazione con Albero di Decisione

1. Un albero di decisione modella tutte le possibili esecuzioni di un algoritmo su un input di dimensione n .
2. Ciascuno degli $n!$ ordinamenti possibili deve apparire in almeno una foglia dell'albero $\Rightarrow L \geq n!$.
3. Un albero binario di altezza h possiede al più 2^h foglie $\Rightarrow 2^h \geq L \geq n!$.
4. Passando al logaritmo in base 2 e applicando l'approssimazione di Stirling ($n! \geq (n/e)^n$):

$$h \geq \log_2(n!) \geq \log_2\left(\frac{n}{e}\right)^n = n \log_2 n - n \log_2 e = \Omega(n \log n)$$

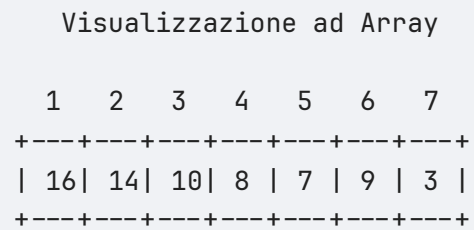
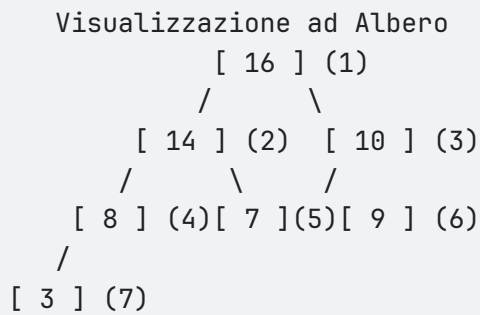
Dato che l'altezza h corrisponde al numero massimo di confronti nel cammino più lungo dalla radice a una foglia, il costo non può scendere al di sotto di $\Omega(n \log n)$.

6. Heaps, HeapSort e Code con Priorità

6.1 Struttura Dati Heap

Un **heap binario** è un albero binario quasi-completo memorizzato in un array contiguo $A[1..n]$.

- $\text{Parent}(i) = \lfloor i/2 \rfloor = i \gg 1$
- $\text{Left}(i) = 2i = i \ll 1$
- $\text{Right}(i) = 2i + 1 = (i \ll 1) | 1$
- **Proprietà di Max-Heap:** $A[\text{Parent}(i)] \geq A[i]$ per ogni nodo $i > 1$. La radice $A[1]$ contiene sempre il massimo assoluto.
- **Altezza dell'Heap:** $\lfloor \log_2 n \rfloor$.



6.2 Operazioni Fondamentali (HeapFy e HeapFyR)

HeapFy / Max-Heapify (Discesa verso il basso)

Ripristina la proprietà di Max-Heap scambiando la chiave corrente con il figlio di valore maggiore e proseguendo ricorsivamente.

```

Algorithm HeapFy(A, j, n):
    k = j
    if (2*j + 1 ≤ n and A[2*j + 1] > A[k]) then
        k = 2*j + 1
    if (2*j ≤ n and A[2*j] > A[k]) then
        k = 2*j
    if (k ≠ j) then
        swap(A[j], A[k])
        HeapFy(A, k, n)

```

- **Complessità:** $T(n) \leq T(2n/3) + O(1) \implies O(\log n)$.

HeapFyR (Ripristino a ritroso / Risalita verso la radice)

Utilizzato per far risalire una chiave verso la radice quando essa è maggiore del proprio genitore:

```

Algorithm HeapFyR(A, j):
    while (j > 1 and A[floor(j / 2)] < A[j]) do
        swap(A[floor(j / 2)], A[j])
        j = floor(j / 2)

```

- **Complessità:** $O(\log n)$.

6.3 Algoritmo HeapSort

```

Algorithm HeapSort(A):
    n = A.length
    // Fase 1: Creazione Max-Heap (Build-Heap bottom-up)

```

```

for i = floor(n / 2) down to 1 do
    HeapFy(A, i, n)

// Fase 2: Estrazione del massimo e ordinamento
for i = n down to 2 do
    swap(A[1], A[i])
    HeapFy(A, 1, i - 1)

```

Dimostrazione del Costo Lineare $O(n)$ di Build-Heap

Un heap di n elementi ha al più $\lceil n/2^{h+1} \rceil$ nodi ad altezza h . Su un nodo ad altezza h , `HeapFy` costa $O(h)$:

$$T(n) = \sum_{h=0}^{\lfloor \log_2 n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O\left(n \sum_{h=0}^{\lfloor \log_2 n \rfloor} \frac{h}{2^h}\right) = O(n \cdot 2) = O(n)$$

- **Complessità Globale di HeapSort:** $\Theta(n) + (n-1) \times O(\log n) = \Theta(n \log n)$ in tutti i casi.
- **In-Place:** Sì ($O(1)$ spazio) | **Stabile:** No.

6.4 Code con Priorità (Priority Queues)

- `Maximum(A)`: restituisce $A[1]$ in tempo $\Theta(1)$.
- `ExtractMax(A)`: estrae $A[1]$, copia l'ultimo elemento nella radice e invoca `HeapFy(A, 1, n-1)` in tempo $O(\log n)$.
- `IncreaseKey(A, i, key)`: assegna $A[i] = key$ e fa risalire il nodo tramite `HeapFyR(A, i)` in tempo $O(\log n)$.
- `Insert(A, key)`: inserisce $-\infty$ in coda e invoca `IncreaseKey` con valore key in tempo $O(\log n)$.

7. Algoritmi di Ordinamento in Tempo Lineare

Gli algoritmi in tempo lineare non utilizzano confronti tra coppie di elementi, ma sfruttano le proprietà intrinseche delle chiavi (es. numeri interi in intervalli limitati).

7.1 Counting Sort

Assume che gli n elementi siano interi appartenenti all'intervallo noto $[0..k]$.

```

Algorithm CountingSort(A, B, n, k):
    let C[0..k] be a new array
    for i = 0 to k do
        C[i] = 0
    // Fase 1: Conta le frequenze di ciascuna chiave
    for j = 1 to n do
        C[A[j]] = C[A[j]] + 1
    // Fase 2: Calcola le frequenze cumulate (prefissi degli indici)
    for i = 1 to k do
        C[i] = C[i] + C[i - 1]
    // Fase 3: Riversa in B scorrendo A da destra a sinistra per preservare la
    stabilità
    for j = n down to 1 do
        B[C[A[j]]] = A[j]
        C[A[j]] = C[A[j]] - 1

```

- **Complessità Temporale:** $\Theta(n + k)$. Se $k = O(n)$, il tempo è strettamente $\Theta(n)$.
- **Spazio Ausiliario:** $\Theta(n + k)$ per i vettori B e C .
- **Stabile:** Sì | **In-Place:** No.

7.2 Radix Sort

Ordina numeri o stringhe di d cifre/caratteri ordinando le cifre partendo dalla **meno significativa** (LSD - *Least Significant Digit*) fino alla più significativa (MSD).

```

Algorithm RadixSort(A, d):
    for i = 1 to d do
        use a stable sort (e.g. CountingSort) to sort array A on digit i

```

Dimostrazione di Correttezza per Induzione

- **Base ($i = 1$):** dopo aver ordinato sulla prima cifra LSD, i numeri sono correttamente ordinati su di essa.
- **Passo Induttivo:** assumiamo che i numeri siano ordinati sulle cifre $1, \dots, i-1$. Quando si ordina sulla cifra i -esima tramite un algoritmo **stabile**:
 1. Se due numeri hanno la cifra i -esima diversa, l'ordinamento sulla cifra i -esima li colloca nell'ordine corretto.
 2. Se due numeri hanno la cifra i -esima uguale, la proprietà di **stabilità** preserva l'ordine relativo precedente (che per ipotesi induttiva era già corretto sulle cifre $1 \dots i-1$).
- **Complessità:** $\Theta(d(n + k))$. Se d è costante e $k = O(n)$, il tempo è $\Theta(n)$.
- **Stabile:** Sì | **In-Place:** No.

7.3 Bucket Sort

Assume che l'input sia distribuito in modo uniforme e indipendente nell'intervallo continuo $[0,1)$.

1. Divide l'intervallo $[0,1)$ in n secchi (*buckets*) di ampiezza $1/n$.
2. Inserisce ciascun elemento $A[i]$ nella lista collegata del secchio $\lfloor n \cdot A[i] \rfloor$.
3. Ordina ogni singolo secchio tramite `InsertionSort`.
4. Concatena i secchi in ordine.

Analisi della Complessità

La dimensione attesa di ciascun secchio è $E[n_i] = 1$. Poiché il costo di `InsertionSort` è $O(n_i^2)$ e $E[n_i^2] = 2 - 1/n$:

$$E[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} O(E[n_i^2]) = \Theta(n) + n \cdot O(1) = \Theta(n)$$

- **Caso Migliore e Medio:** $\Theta(n)$.
- **Caso Peggior:** $\Theta(n^2)$ (se tutti gli elementi cadono nello stesso secchio).
- **Stabile:** Sì | **In-Place:** No.

7.4 Tabella Comparativa degli Algoritmi di Ordinamento

Algoritmo	Tempo Migliore	Tempo Peggior	Tempo Medio	Spazio Ausiliario	Stabile?	In Place?
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Sì	Sì
Reiterate / Selection	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	No	Sì
Bubble Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Sì	Sì
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$	Sì	No
QuickSort	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(n \log n)$	$O(\log n)$	No	Sì
HeapSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(1)$	No	Sì
Counting Sort	$\Theta(n + k)$	$\Theta(n + k)$	$\Theta(n + k)$	$\Theta(n + k)$	Sì	No

Algoritmo	Tempo Migliore	Tempo Peggior	Tempo Medio	Spazio Ausiliario	Stabile?	In Place?
Radix Sort	$\Theta(d(n+k))$	$\Theta(d(n+k))$	$\Theta(d(n+k))$	$\Theta(n+k)$	Sì	No
Bucket Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n)$	$\Theta(n)$	Sì	No

8. Statistica d'Ordine e Selezione per Rango

Dato un insieme A di n numeri distinti e un intero $i \in [1..n]$, l'obiettivo è determinare l' i -esimo elemento più piccolo (elemento con rango i).

- $i = 1$: Minimo.
- $i = n$: Massimo.
- $i = \lfloor (n+1)/2 \rfloor$: Mediana.

8.1 QuickSelect (Randomized-Select)

QuickSelect sfrutta il partizionamento di QuickSort, ma scende in **un solo sotto-array**:

```
[  Elementi < Pivot   | Pivot |   Elementi > Pivot   ]
p                    q-1  q  q+1                      r
<----- k elementi ----->
```

```
Se i = k: Trovato! Ritorna A[q]
Se i < k: Cerca rango i nel sotto-array sinistro [p .. q-1]
Se i > k: Cerca rango (i - k) nel sotto-array destro [q+1 .. r]
```

```
Algorithm QuickSelect(A, p, r, i):
    if p = r then
        return A[p]
    q = RandomizedPartition(A, p, r)
    k = q - p + 1    // Numero di elementi nel sotto-array sinistro compreso il
pivot
    if i = k then
        return A[q]
    else if i < k then
        return QuickSelect(A, p, q - 1, i)
    else
        return QuickSelect(A, q + 1, r, i - k)
```

Analisi della Complessità

- **Caso Migliore:** $\Theta(n)$ ($T(n) = T(n/2) + \Theta(n) \implies \Theta(n)$).
- **Caso Peggior:** $\Theta(n^2)$ (partizioni sbilanciate $T(n) = T(n-1) + \Theta(n)$).
- **Caso Medio:** $\Theta(n)$ tempo atteso. Trattando la scelta di un buon perno come una prova di Bernoulli con probabilità di successo $\geq 1/2$, il numero atteso di prove prima del successo è ≤ 2 (distribuzione geometrica), garantendo tempo lineare medio.

8.2 Selezione in Tempo Lineare nel Caso Peggior: Mediana delle Mediane (BFPT)

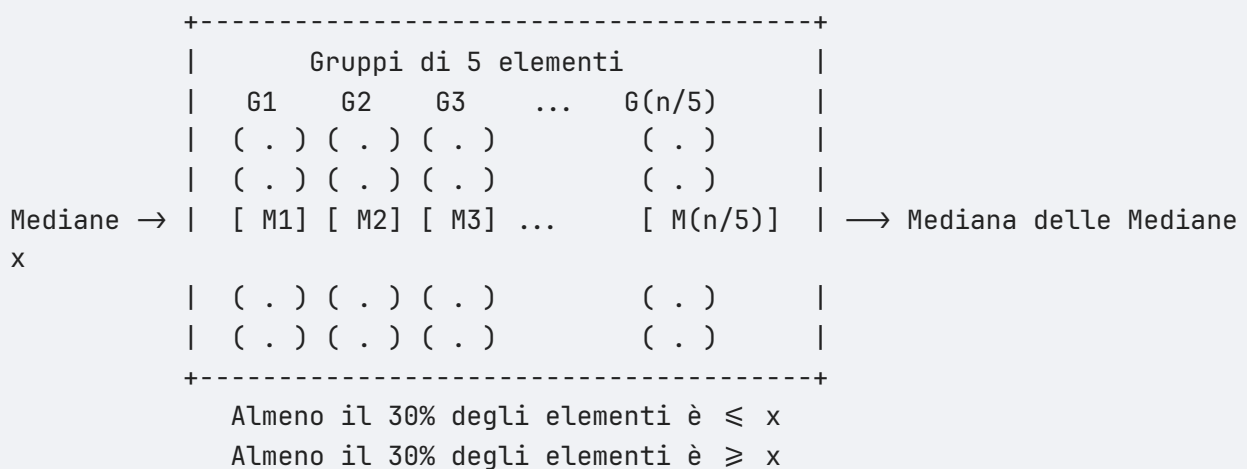
L'algoritmo deterministico **SELECT** (Blum, Floyd, Pratt, Rivest, Tarjan) trova l' i -esimo elemento garantendo tempo $O(n)$ anche nel caso peggior assoluto scegliendo un pivot bilanciato.

Procedura in 5 Fasi

1. Suddivide gli n elementi in $\lceil n/5 \rceil$ gruppi da 5 elementi (più un eventuale gruppo finale con $n \bmod 5$ elementi).
2. Determina la mediana di ciascuno dei $\lceil n/5 \rceil$ gruppi ordinando ogni gruppetto da 5 elementi con ≤ 7 confronti ($O(1)$ a gruppo).
3. Trova ricorsivamente la mediana delle mediane x tramite chiamata:

$$x = \text{SELECT}(\text{Mediane}, 1, \lceil \frac{n}{5} \rceil, \lceil \frac{1}{2} \lceil \frac{n}{5} \rceil \rceil)$$

4. Partiziona l'array iniziale attorno al pivot x tramite **Partition**. Sia k il rango di x .
5. Se $i == k$ ritorna x ; se $i < k$ invoca ricorsivamente **SELECT** sulla porzione sinistra di rango i ; se $i > k$ invoca **SELECT** sulla porzione destra di rango $i - k$.



Dimostrazione del Costo Lineare nel Caso Peggior

Almeno la metà delle $\lceil n/5 \rceil$ mediane è $\leq x$, e ciascuno dei corrispondenti gruppi contiene almeno 3 elementi $\leq x$. Il numero di elementi $\leq x$ è almeno:

$$3(\lceil \frac{1}{2} \lceil \frac{n}{5} \rceil \rceil - 2) \geq \frac{3n}{10} - 6$$

Nel caso peggiore, la chiamata ricorsiva al passo 5 viene invocata su al più $n - (3n/10 - 6) = \frac{7n}{10} + 6$ elementi.

La relazione di ricorrenza è:

$$T(n) \leq T(\lceil \frac{n}{5} \rceil) + T(\frac{7n}{10} + 6) + O(n)$$

Poiché la somma delle frazioni è strettamente minore di 1:

$$\frac{n}{5} + \frac{7n}{10} = \frac{9n}{10} < n$$

Dimostrando per induzione con $T(n) \leq cn$:

$$T(n) \leq c\frac{n}{5} + c(\frac{7n}{10} + 6) + dn = \frac{9}{10}cn + 6c + dn = cn - (\frac{cn}{10} - 6c - dn)$$

Per $n \geq 70$, ponendo $c \geq 20d$, la parentesi è ≥ 0 , dimostrando che $T(n) \leq cn = O(n)$.

- **Caso Peggiore Assoluto:** $\Theta(n)$.
- **Spazio Ausiliario:** $\Theta(n)$.